

# 回文树

## 回文树

回文树的关键技术

1. 后缀链跳跃
2. 奇偶字典树
3. 建回文树
4. Fail指针
5. 复杂度分析

模板代码

例 Victor and string(hdu 5421)

回文串算法在现代工程领域中有着广泛的应用，在生物信息学中，它是基因编辑技术的基础——用于识别DNA限制性酶切位点、筛选基因编辑靶点；网络安全领域，可以在入侵检测系统中匹配异常流量的回文特征；自然语言处理中可以用于中文分词歧义消解；工业物联网中，通过分析振动信号的回文模式提前预警设备故障；此外还广泛应用于数据压缩、密码学哈希碰撞检测等场景。

Manacher算法解决了求最长回文子串的问题，下面考虑一个一般性的回文串问题：求一个字符串 $S$ 中的所有回文串。用Manacher算法可以求解，先求出 $P[]$ 数组， $P[i]$ 对应的就是以 $S[i]$ 为中心的一个最长回文串，统计所有 $P[i]$ 即可。但是，如果问题复杂一些，如本节的例题hdu 5421，动态在字符串 $S$ 的首尾加入字符，然后在线求所有回文串，这种情况无法使用Manacher算法。

回文串问题有很多，有没有一种通用且高效的算法？这就是回文树，它具有以下优点。

- (1) 效率高，时间复杂度为 $O(n)$ 。
- (2) 代码简短。
- (3) 支持在线查询和更新。

回文树的缺点是空间复杂度较差，比Manacher这样的专用算法使用的空间大。

## 回文树的关键技术

回文树的关键技术可以概括为“奇偶字典树+后缀链跳跃”。下面循序渐进地给出这种巧妙方法的思维过程。

后缀(Suffix)：一个字符串的一个后缀是指从某个位置开始到末尾的一个子串。

### 1. 后缀链跳跃

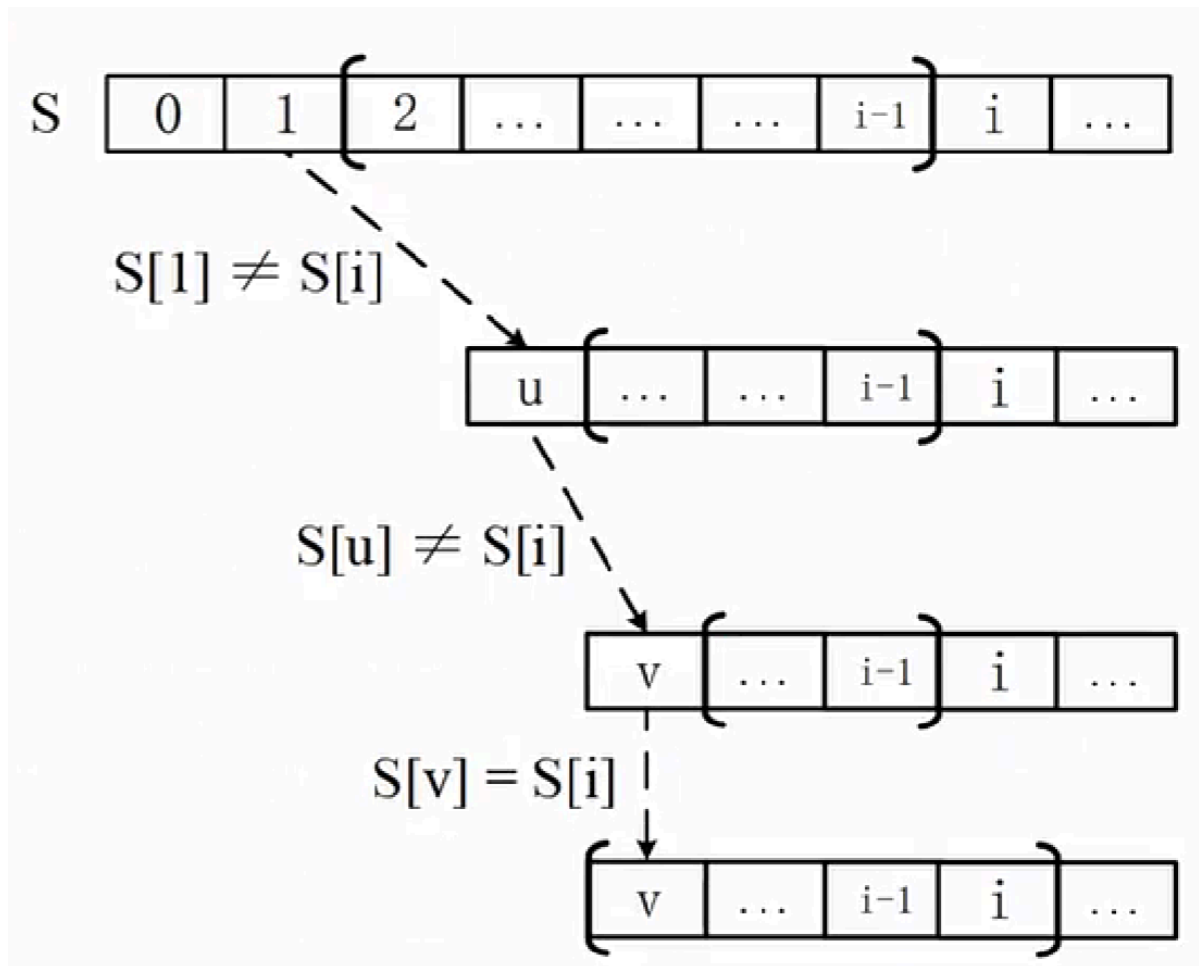
如何设计一种高效的求 $S$ 的所有回文串的算法？很多字符串算法有一个相似点：在已经求得 $S[0] \sim S[i-1]$ 结果的基础上，递推到 $S[0] \sim S[i]$ 的解，从而实现线性的 $O(n)$ 复杂度的算法。例如，Manacher算法是从 $P[0] \sim P[i-1]$ 递推到 $P[i]$ 。

观察字符串 $S$ 中的回文串，思考是否有这种递推的方法。例如， $S = \text{”abcba”}$ ，从”abc”递推到”abcb”时，检查最后的’b’是否与前面的”abc”形成回文；再递推到”abcba”，检查最后的’a’是否与前面的”abcb”形成回文。有以下观察结果。

- (1) 如果末尾的新字符能产生回文，一定是与前面的后缀一起。例如，”abc”后插入’b’，是与前面的后缀”bc”一起形成了回文”bcb”；再如，”abcb”后插入’a’，是与前面的后缀”abcb”一起形成回文”abcba”。
- (2) 若末尾新字符能产生回文，则前面的对称位置一定有一个相同的字符。例如，”abcb”后插入’a’形成回文”abcba”，最后的’a’与第1个’a’相同。

(3) 在这两个相同的字符之间，也是一个回文。例如，"abcba"中间的"bcb"也是回文。

(4) 末尾新插入字符后，应该从前面尽量远的地方检查，才能更快找到回文串。例如，"abcb"后新插入'a'，得到的回文串是最远的"abcba"。但是这个"尽量远"，不能从开头开始逐个检查，否则就变成暴力的中心扩展法了。这个"尽量远"显然是从前面的最长后缀开始，即"bcb"的前面。



上图演示了递推计算过程。方括号内是回文串，也是 $S[i]$ 之前的后缀。虚线箭头演示了查找的过程，从 $S[i]$ 前面最长的回文串(后缀)开始查找，最后找到 $S[v] = S[i]$ ， $S[v] \sim S[i]$ 是一个新的回文串，而且是包含 $S[i]$ 的最长回文串。设 $S[i]$ 前面的后缀的长度为 $len$ ，那么应该比较 $S[i]$ 和 $S[i - len - 1]$ 是否相同。称这个操作为"后缀链跳跃"，它是跳跃前进的，速度很快。

## 2. 奇偶字典树

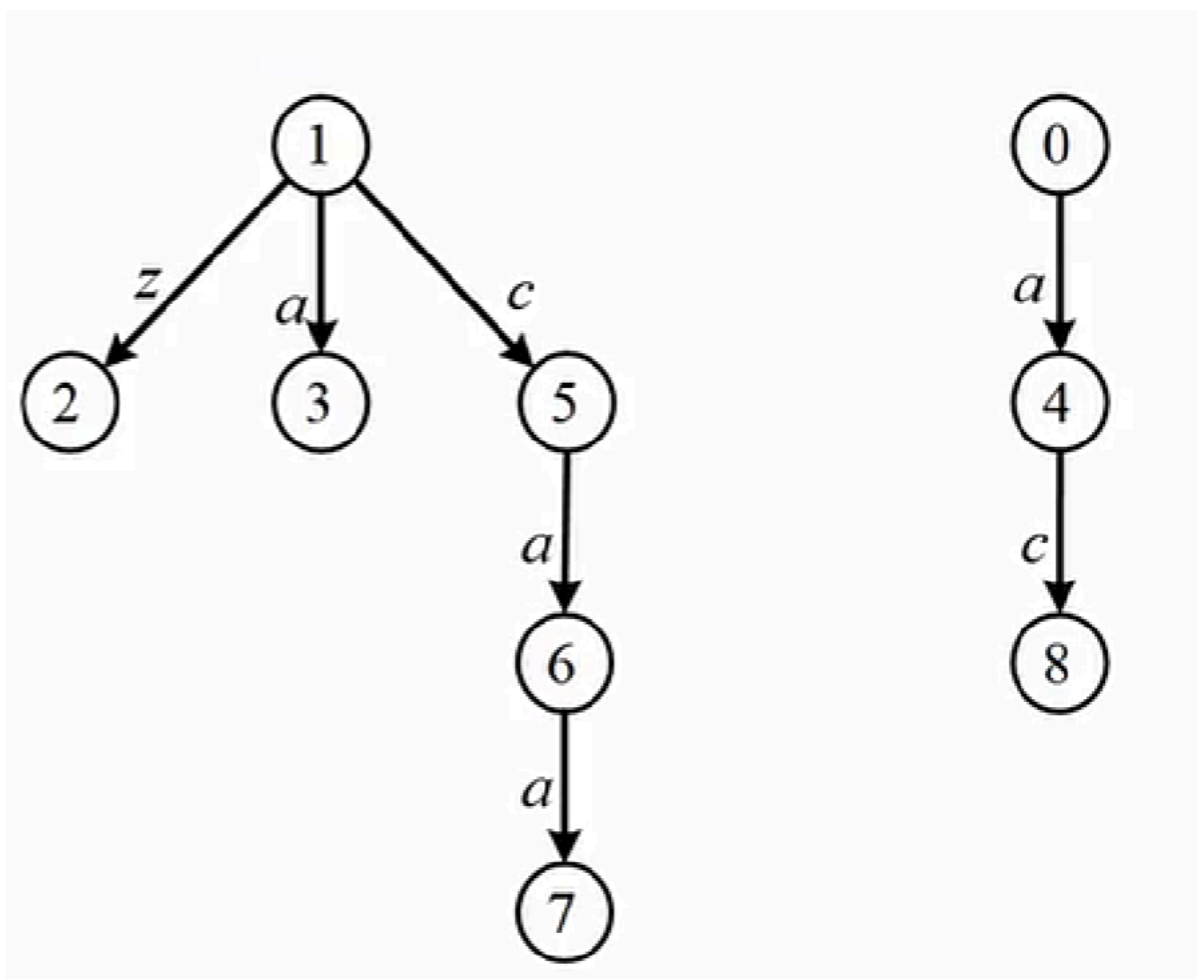
总结以上观察结果，其核心是记录回文串(后缀)，并能做"后缀链跳跃"。用回文树这种数据结构能完美地实现，其要点如下。

(1) 用字典树存储回文串。这些回文串也是前文提到的后缀，用于从 $S[0] \sim S[i - 1]$ 递推到 $S[i]$ 。字典树是一种高效率存储字符串的数据结构，能帮助提高访问速度。

(2) 回文串(后缀)之间有包含关系。回顾字典树的存储方式，每个节点是一个字符，从根到一个子节点的链路，存储了一个字符串。这条链路上的节点正好体现了字符串的包含关系。

(3) 用一个辅助数据(虚线箭头表示的后缀链)定位到新的回文串，即包含 $S[i]$ 的最长回文串。然后把这个新回文串也存储到字典树上。

以 $S = \text{"zaacaac"}$ 为例图示回文树。

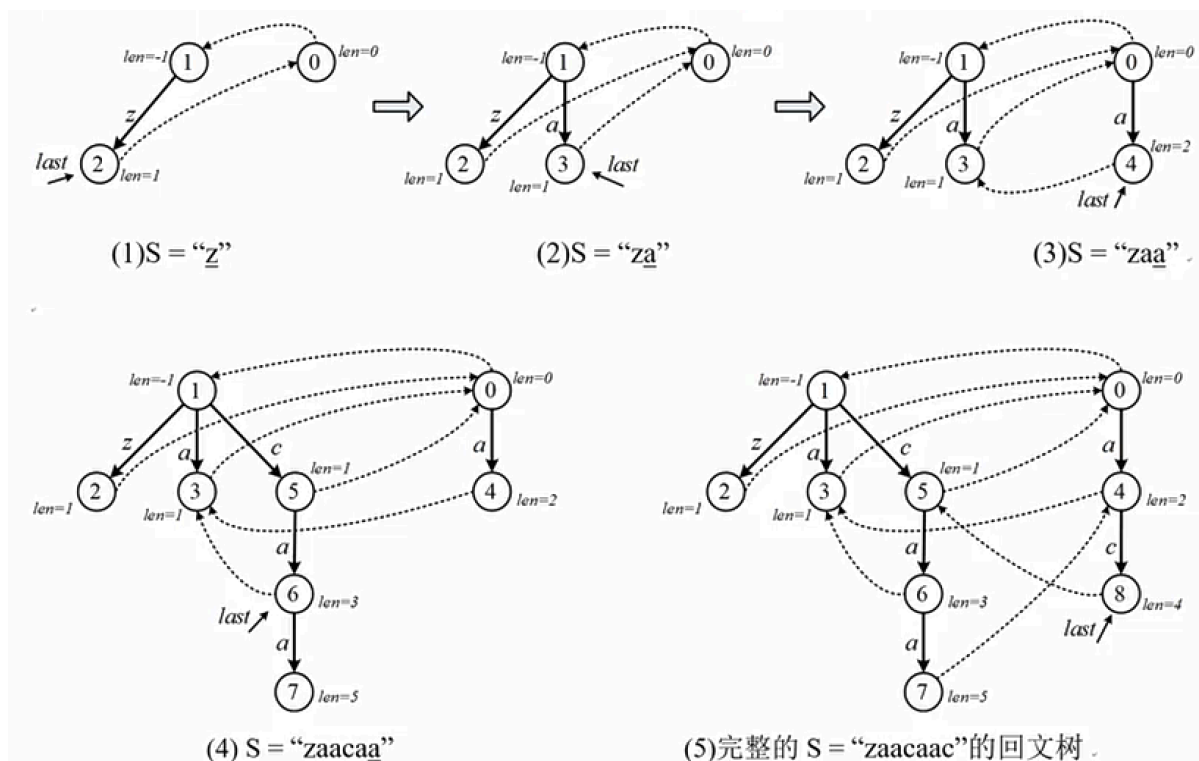


回文树包括奇、偶两棵字典树。前面提到按长度分有奇回文串和偶回文串两种情况，为了简化问题，用两棵字典树分别存储，0节点是偶回文串的根，1节点是奇回文串的根。

任意一个子节点都表示一个不同的回文串，从根出发到一个子节点的一条链路存储一个回文串的一半。读一条链上回文串的方法是从子节点向上读到根，再从根向下读回到子节点，如果是1号根，最靠近1的边只读一次，如果是0号根，靠近0号根的边读两次。这样就解决了奇回文串和偶回文串的存储问题。上图中，1号根字典树存奇回文串：2号点是回文串'z'，3号点是'a'，5号点是'c'，6号点是'aca'，7号点是'aacaa'；0号根字典树存偶回文串：4号点是'aa'，8号点是'caac'。共7种不同的回文串，存储在7个子节点上。这两棵字典树将用"后缀链"连接成一棵回文树。

### 3. 建回文树

如何建这棵回文树？建树的过程是按顺序逐个处理 $S$ 的字符的过程，也就是从 $S[0] \sim S[i-1]$ 递推到 $S[i]$ 求回文串的过程。由于每个节点表示一个不同的回文串，那么每处理一个 $S[i]$ ，最多只会在回文树上增加一个节点。可以推论出，长度为 $n$ 的字符串 $S$ ，最多有 $n$ 个不同的回文串。



上图给出了建回文树的过程。初始时，回文树有0号和1号点，然后逐个加入 $S$ 的字符，建树的过程就是求解回文串的过程。

图中的边有以下两种。

(1) 带权有向边，用实线箭头表示，它的作用是表达回文串。边的起点为 $u$ ，终点为 $v$ ，边的权值是字符 $x$ ，在起点 $u$ 的前后加上字符 $x$ ，得到终点 $v$ 。例如，图(d)中，5号点是起点 $u = 'c'$ ，6号点是终点 $v$ ，边的权值 $x = 'a'$ ，则6号点 $v = 'a' + 'c' + 'a' = 'aca'$ 。

(2) 后缀链，用虚线箭头表示，是无权边，它们连接了两棵字典树。其作用即是前面图中的虚线箭头，用于查找新加入字符前面的回文串(后缀)。在代码中定义虚线箭头为 $Fail$ 指针。

为了逐个处理 $S$ 的字符，代码中用 $last$ 变量记录上次处理到 $S$ 的第几个字符，具体存储位置是 $tree[last]$ 。

设已经处理好了 $S[0] \sim S[i-1]$ ，现在处理 $S[i]$ 。首先根据 $last$ 找到前一个字符 $S[i-1]$ 在回文树上的位置，然后执行“后缀链跳跃”过程。设 $S[i-1]$ 的最长后缀 $X$ 的长度为 $len$ ，比较 $S[i]$ 和 $S[i-len-1]$ 是否相同，分以下两种情况。

(1)  $S[i]$ 和 $S[i-len-1]$ 相同。此时 $S[i-len-1] + X + S[i]$ 是一个新的回文串，把 $S[i]$ 挂到回文树上 $S[i-1]$ 的子节点上。如图(d)所示，检查到 $S = 'zaacaac'$ 中的 $S[5] = 'a'$ 时，前一个回文 $X = 'aca'$ 存在于 $last = 6$ 号点上，而 $S[5]$ 和 $S[i-len-1] = S[5-3-1] = S[1] = 'a'$ 相等，则 $S[i-len-1] + X + S[i] = 'a' + 'aca' + 'a' = 'aaca'$ 是一个新的回文。建立一个新节点7，把7号点挂在6号点的子节点上。

(2)  $S[i]$ 和 $S[i-len-1]$ 不同。此时 $S[i-len-1] + X + s[i]$ 不是回文串，需要缩小并查找一个新的后缀 $X$ 。如图(e)所示，检查到 $S = 'zaacaac'$ 中的 $S[6] = 'c'$ 时，前一个回文 $X = 'aaca'$ 存在于 $last = 7$ 号点上，但是 $S[6]$ 和 $S[i-len-1] = S[6-5-1] = S[0] = 'z'$ 不等，不能构成新的回文。查找新的后缀，即沿着7号点的后缀链找到4号点，4号点表示回文“aa”。这次 $S[6]$ 能与4号点构成新的回文“caac”，新建节点8，并把8号点挂在4号点的子节点上。

请根据上述解释自己分析图(a)~图(c)。

图中节点上的 $len$ 是回文串的长度， $len$ 用于前文提到的比较 $S[i]$ 和 $S[i - len - 1]$ 是否相同。例如，6号点表示回文串"aca"，它的 $len = 3$ 。奇字典树上的 $len$ 是奇数，偶字典树上的 $len$ 是偶数。编码时用到一个小技巧，置1号点的 $len = -1$ ，0号点的 $len = 0$ ，这样所有实线上边的父子节点的 $len$ 相差2。父子节点的 $len$ 相差2，是因为子节点表示的回文串是在父节点的两侧各扩展一个字符。

## 4. Fail指针

前面提到，定义虚线箭头为 $Fail$ 指针。这里讨论一个关键问题：如何给新节点的 $Fail$ 指针赋值？ $Fail$ 指针指向它的最长回文后缀，建立了后缀链。例如，前面的图(d)中，把7号点挂在6号点下面后，让7号点的 $Fail$ 指针指向4号点，4号点"aa"是7号点"aacaa"的最长回文后缀。

一个节点(设这个节点的新字符为 $Y$ )的最长回文后缀，是在它父节点的某个回文后缀两侧各拓展一个 $Y$ 得到的，那么在新建一个节点之后，可以沿着它的父节点的后缀链找，直到跳到第1个两侧能拓展这个字符 $Y$ 的节点(这个节点有 $Y$ 儿子)为止，这个 $Y$ 儿子就是新建节点的最长回文后缀。

7号点表示回文"aacaa"，它的新字符是' $a$ '。7号点的父亲是6号点，沿着6号点的后缀链一直找到0号点，0号点有一个等于' $a$ '的子节点4，让7号点的 $Fail$ 指针指向4号点。

## 5. 复杂度分析

回文树的时间复杂度如何？回文树的建立是通过逐一处理 $S$ 的每个字符得到的，最多加入 $n$ 个节点，所以复杂度为 $O(n)$ 。加入一个节点最耗时的操作是搜索后缀链(模板代码中的 $get\_fail()$ 函数)，这与回文树的形态有关。若回文树都是短链，即 $S$ 中几乎没有长度大于1的回文串， $get\_fail()$ 函数复杂度为 $O(1)$ ；若回文树有长链， $get\_fail()$ 函数计算量可能较大，但仍是一个很小的数。总体上看回文树的时间复杂度为 $O(n)$ 。

提示

回文树的空间复杂度较差，因为它本质上是字典树，所以也继承了字典树较差的空间复杂度。

## 模板代码

下面给出一道模板题，在字符串的首尾扩展新字符，统计回文串的数量。

### 例 Victor and string(hdu 5421)

**问题描述：**字符串 $S$ 初始为空，做 $n$ 个操作。

操作1：在 $S$ 的头部加一个字符 $c$ ；

操作2：在 $S$ 的尾部加一个字符 $c$ ；

操作3：查询不同回文串的数量；

操作4：查询所有回文串的数量。

**输入：**有很多测试，每个测试的第1行输入 $n$ ，后面有 $n$ 个操作。 $1 \leq n \leq 100000$ 。

**输出：**对于操作3和操作4，输出结果。

虽然前面对回文树的解释看起来比较复杂，但是代码很短。以下代码在前面已经有详细解释。

```
1 //改写自 https://blog.csdn.net/qq\_40858062/article/details/103956999
2 #include <iostream>
3 using namespace std;
4 typedef long long ll;
5 const int N = 3e5 + 8;
6 int s[N];
```

```

7  struct node{
8      int len,fail,son[26],siz;
9      void init(int _len){
10         memset(son,0,sizeof(son));
11         fail = siz = 0;
12         len = _len;
13     }
14 }tree[N];
15 ll num,last[2],ans,L,R;          //L:在S的头部加字符; R:在S的尾部加字符
16 void init(){                     //初始化一个节点
17     last[0] = last[1] = 0;        //从0号点开始
18     ans = 0; num = 1;
19     L = 1e5 + 8, R = 1e5 + 7;
20     tree[0].init(0);              //小技巧,置0号点len = 0
21     memset(s, -1, sizeof(s));
22     tree[1].init(-1);             //小技巧,置1号点len = -1
23     tree[0].fail = 1;             //0指向1,1不必指向0
24 }
25 int getfail(int p,int d){         //后缀链跳跃,复杂度可以看作O(1)
26     if(d)                         //新字符在尾部
27         while(s[R - tree[p].len - 1] != s[R])
28             p = tree[p].fail;
29     else                           //新字符在头部
30         while(s[L + tree[p].len + 1] != s[L])
31             p = tree[p].fail;
32     return p;                     //返回节点p
33 }
34
35 void Insert(int x,int d){          //向回文树上插入新节点,这个节点表示一个新回
文串
36     if(d) s[++R] = x;             //新字符x插到S的尾部
37     else s[--L] = x;              //新字符x插到S的头部
38     int father = getfail(last[d],d); //插到一个后缀的子节点上
39     int now = tree[father].son[x]; //字典树上还没有这个字符,新建一个
40     if(!now){
41         now = ++num;
42         tree[now].init(tree[father].len + 2);
43         tree[now].fail = tree[getfail(tree[father].fail,d)].son[x];
44         tree[now].siz = tree[tree[now].fail].siz + 1;
45         tree[father].son[x] = now;
46     }
47     last[d] = now;
48     if(R - L + 1 == tree[now].len) last[d^1] = now;
49     ans += tree[now].siz;
50 }
51 //char ch = x + 'a';              //在这里打印回文树,帮助理解
52 //cout<<" fa = "<<father<<", me = "<<now<<", char = "<<ch;
53 //cout<<", fail = "<<tree[now].fail<<", len = "<<tree[now].len<<endl;
54 //}
55 int main(){
56     int op,n;
57     while(scanf("%d",&n)!=EOF){
58         init();
59         while(n--){
60             char c; scanf("%d",&op);
61             if(op == 1) scanf("%c",&c), Insert(c - 'a',0);

```

```
62         if(op == 2) scanf(" %c",&c), Insert(c - 'a',1);
63         if(op == 3) printf("%lld\n",num - 1);
64         if(op == 4) printf("%lld\n",ans);
65     }
66 }
67 return 0;
68 }
```